Assignment 4 Facade Pattern

Option A Media and File Conversion Facade

Student: Timur Naumov
Group: SE-2537
Course: ShP-2216 Software Design Patterns

1 Introduction

This project implements a MediaConversionFacade in Java 17. A client supplies a file name and a target format through one convert method. The Facade coordinates metadata reading, codec selection, video bitrate conversion, and audio normalization in the correct order. This follows the Facade structure presented in Lecture 6: the client uses a simple interface, while the subsystem classes perform the individual operations.

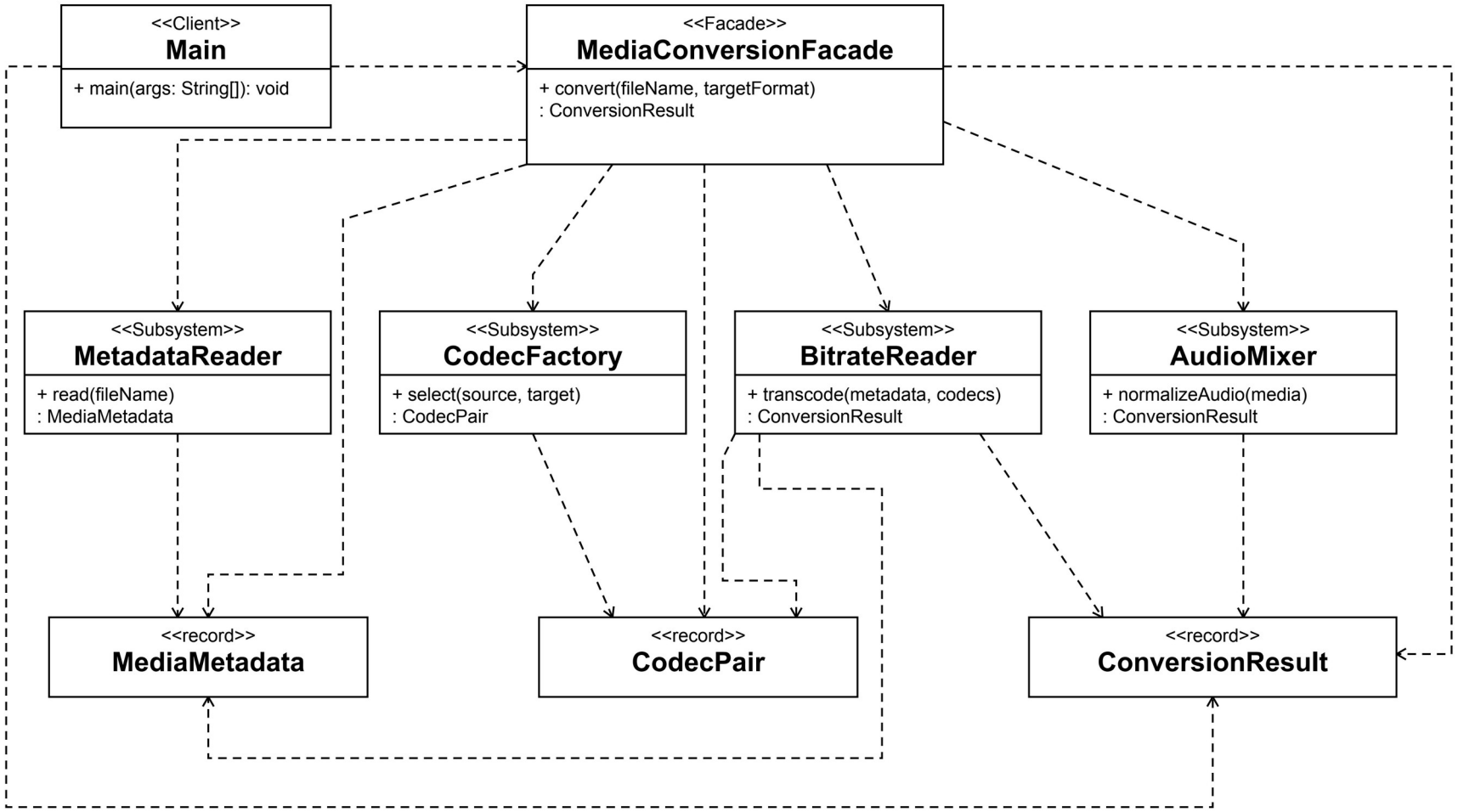
The program uses simulated media data. It does not decode or create a real media file. Each subsystem still performs a concrete step: it validates input, selects settings, transforms bitrate values, or recalculates the output size.

Conversion steps

1. MetadataReader validates the name, extracts the extension, and returns fixed sample metadata: 120 seconds, 4,500 kbps video, 192 kbps audio, and a -4.0 dB audio peak.
2. CodecFactory accepts AVI, MKV, and MP4 sources, and MP4 or WebM targets. It selects Xvid or H.264 decoding and H.264 or VP9 encoding.
3. BitrateReader caps the simulated video bitrate at 2,500 kbps for MP4 or 1,800 kbps for WebM and estimates the output size.
4. AudioMixer caps the audio bitrate at 160 kbps, sets the simulated peak to -1.0 dB, and recalculates the size.

For sample.avi converted to MP4, the final estimate is ceil((2500 + 160) × 120 / 8) = 39,900 kB. This estimate excludes container overhead.

2 UML class diagram



Dashed open arrow = dependency. The arrow points to the used class. Only the main operations are shown.

3 Clean Code principles

3.1 Meaningful names

Names describe the domain action and its data. The variables metadata, codecs, and convertedMedia show what is passed between the conversion stages. This annotated excerpt is taken from MediaConversionFacade.

```
MediaMetadata metadata = metadataReader.read(fileName);
CodecPair codecs = codecFactory.select(metadata.sourceFormat(), targetFormat);
ConversionResult convertedMedia = bitrateReader.transcode(metadata, codecs);
```

3.2 Single Responsibility

The Facade coordinates the workflow. It delegates metadata validation, codec rules, bitrate calculations, and audio settings to their own classes. Its final step delegates audio normalization rather than implementing that calculation again.

```
ConversionResult convertedMedia = bitrateReader.transcode(metadata, codecs);
return audioMixer.normalizeAudio(convertedMedia);
```

3.3 A small public interface

The client provides two ordinary strings and receives an immutable result. It does not construct subsystem objects or choose their call order. Codec names in the result are diagnostic data; the API does not expose subsystem instances.

```
public ConversionResult convert(String fileName, String targetFormat)

// Client call in Main
ConversionResult result = facade.convert("sample.avi", "mp4");
```

The Facade creates its helper objects locally. UML therefore models these relationships as dependencies. The design does not store or own subsystem objects through fields.

3.4 Focused subsystem classes

AudioMixer handles only the simulated audio settings and their effect on estimated size. CodecFactory handles supported formats and codec selection. The record types carry immutable data between these focused operations.

```
int audioBitrateKbps = Math.min(media.audioBitrateKbps(), MAX_AUDIO_BITRATE_KBPS);
long estimatedSizeKb = estimateSizeKb(
        media.videoBitrateKbps(), audioBitrateKbps, media.durationSeconds());
return media.withAudioSettings(
        audioBitrateKbps, NORMALIZED_AUDIO_PEAK_DB, estimatedSizeKb);
```

3.5 Explicit error handling

Invalid requests fail with IllegalArgumentException and a message identifying the problem. The program rejects null or blank names, missing extensions, and unsupported source or target formats. It does not silently select a replacement format.

```
if (fileName == null || fileName.isBlank()) {
    throw new IllegalArgumentException("File name must not be null or blank.");
}
```

4 Conclusion

The Facade hides object creation, the order of conversion steps, and data handoff between the subsystems. The client needs only convert(fileName, targetFormat). All four subsystem classes remain public and directly usable by advanced clients, and none refers back to the Facade. The conversion is a deterministic simulation suitable for a live demonstration.

The project compiles with the Java 17 language target. Acceptance checks cover the sample result, case-insensitive extensions, and invalid input. The demo returns 2,500 kbps video, 160 kbps audio, a -1.0 dB peak, and a 39,900 kB estimate.

5 GitHub repository

https://github.com/tnaumovv/assignmen4-Facade-Pattern

References

1. Assignment #4 Facade Pattern. ShP-2216 Software Design Patterns assignment instructions.
2. Almas Ospanov. Lecture 6 The Facade Design Pattern. Course lecture slides, especially the structure and video conversion examples.